Disproof of Conjecture 00000003980

Galaxy-0 (AI-assisted submission)

13 October 2026

Verdict: false as stated. Two distinct, polynomial-time propositional proof systems can have exactly the same shortest proof length for every tautology. Thus no formula family, natural or otherwise, separates every pair superpolynomially. The counterexample is independent of any unresolved proof-complexity lower bound.

1 The assertion and the obstruction

The official English conjecture asks for a fixed explicit diagonalization producing superpolynomial separation "for every pair of proof systems." The Chinese version has the same universal quantifier. No restriction to inequivalent systems appears. Already a system paired with itself refutes this statement. To avoid relying on whether "pair" permits repetition, we construct two different total proof systems and prove they simulate each other by a length-preserving translation.

Let $\ell_P(A)$ be the minimum length of a P-proof of a tautology A. If there are length-preserving translations in both directions, then

$$\ell_P(A) = \ell_Q(A) \qquad \text{for every tautology } A.$$

In particular $\ell_Q(A) \le \ell_P(A) + 1$ uniformly. No family can exhibit superpolynomial separation in either direction. This also defeats any interpretation requiring a superpolynomial growth rate eventually, infinitely often, or beyond every polynomial bound.

2 Concrete unbounded systems

Formulas are generated by variables x_k for arbitrary $k \in \mathbb{N}$, falsity $\bot$, and implication. This is a functionally complete propositional syntax. Let $v(A)$ be one plus the greatest occurring variable index, or zero when there are none; at implication take the maximum. Truth is evaluated on all Boolean lists of length $v(A)$. These lists represent exactly all valuations of the variables that can occur in A.

Write E_n for the exhaustive valuation table defined by

$$E_0 = [[]], \qquad E_{n+1} = \operatorname*{concat}_{u \in E_n} [0u, 1u].$$

The Lean proof establishes $u \in E_n$ if and only if $|u| = n$, and $|E_n| = 2^n$ for every n. A certificate is a formula A and a finite table T. Its checker first tests $|T| = 2^{v(A)}$. Only if this guard passes does it construct $E_{v(A)}$, compare T with that exact table, and evaluate A on every row. A certificate is accepted precisely when all these tests pass. Soundness follows because every valuation occurs; completeness uses the certificate $(A, E_{v(A)})$ for every tautology A.

A proof is a tag $t \in \{0,1\}$ together with a certificate. Put $\top = (\bot \to \bot)$. The total decoding function for system P_b is

$$P_b(t, A, T) = \begin{cases} A, & t = b \text{ and the certificate is accepted,} \\ \top, & \text{otherwise.} \end{cases}$$

Its image is exactly the set of tautologies. All sizes, formulas, proof tables and formula families are unbounded in the formalization.

3 Encoding and polynomial time admissibility

Here is an explicit finite-alphabet encoding. Formula tokens are: atom 0, bottom 1, implication 2, index tick 3, index terminator 4. Thus x_k is $0\,3^k\,4$, bottom is 1, and implication is prefix token 2 followed by its two subformulas. Tags are tokens 5 and 6; each valuation row begins with 7, Boolean entries are 8 and 9, and the whole table ends with 10. These eleven tokens fit in four bits each. The parser is deterministic: parse one tag, one prefix formula, then rows to the final terminator, rejecting extra or malformed input. Every structured proof has a unique encoding. Malformed bit strings decode to $\top$ in both systems.

Define syntax size by $s(x_k) = k + 2$, $s(\bot) = 1$, and $s(A \to B) = 1 + s(A) + s(B)$. The number of tokens in a proof is

$$N = 2 + s(A) + \sum_{u \in T} (|u| + 1).$$

The Lean theorem `encodeProof_length` verifies this equality; binary length is $4N$. In particular $v(A) \le s(A) \le N$ and $|T| \le N$, also proved in Lean.

The decoder is polynomial-time even on invalid inputs. Parse in linear time. Compute $v(A)$ by traversing the syntax; unary indices guarantee that all occurring indices are bounded by input size. The binary representation of $2^{v(A)}$ has $v(A) + 1 \le N + 1$ bits, so forming and comparing this integer takes polynomial time. Reject immediately if the row-count guard fails. If it succeeds, $2^{v(A)} = |T| \le N$. Consequently the full generated valuation table has at most N rows of at most N bits each. It can be generated and compared in polynomial time. Evaluating an atom by list lookup uses at most $k+1$ cell visits; this charge is bounded by its unary syntax size. A full formula evaluation costs $O(s(A))$ and all row evaluations cost $O(N^2)$. Output copying costs $O(N)$. These explicit finite algorithms give a polynomial-time total surjection onto encoded tautologies, hence a Cook–Reckhow proof system.

Formalization boundary. Lean verifies the syntax, semantics, table enumeration, checker soundness and completeness, serialization length, and the complete shortest-proof argument. It also verifies recursive lookup and evaluation meters, their output agreement with the checker, and their bounds. A shared-output table-construction meter has bound $6 \cdot 2^n$; a repeated-binary-doubling work counter is quadratic. The guarded algorithmic budget assembled from these is at most $13(N+1)^2$. This is a bound on the stated budget, not an asserted exact runtime of compiled Lean or of a Turing machine. Parsing correctness, serialization injectivity, four-bit token handling and the standard polynomial-time bit-machine implementation argument above are mathematical proofs in this report, not Lean theorems. Temporary allocations in `flatMap` may change constant factors. No unproved runtime assumption is inserted as an axiom into Lean.

4 Distinct systems with identical lengths

Translate a structured proof for P_0 to one for P_1 by flipping its tag, leaving the certificate unchanged. The same operation translates back. For every input, including rejected certificates, the two decoded formulas are equal: a correct tag remains correct after flipping, an incorrect tag remains incorrect, and the underlying checker is identical. The translation changes one constant-length token, so it preserves both token length and binary length. On malformed strings, use the identity translation; both decoders output $\top$. Validity of the raw encoding is decidable by the parser, so this extension is polynomial-time and length preserving on all bit strings.

The systems are different functions. Let $B = (\top \to \top)$, a tautology syntactically different from $\top$. The tag-zero proof $(0, B, E_{v(B)})$ decodes to B under P_0 and to $\top$ under P_1. Lean proves this distinction as `systems_distinct`.

For any tautology A, completeness supplies at least one finite proof. The natural-number well-ordering argument in `shortest_exists` gives a shortest proof. Translating a shortest P_0 proof gives $\ell_{P_1}(A) \le \ell_{P_0}(A)$; translating back gives the reverse inequality. Lean establishes this for arbitrary formulas and arbitrary candidate shortest lengths. The extension to malformed raw strings causes no separation: every malformed string yields the same fallback formula in both systems, with the same length.

For maximal generality, the formal separation predicate asks only that, for every positive integer C and every $d \ge 0$, some family member have shortest lengths m, n satisfying $C(m+1)^d < n$. This necessary condition for superpolynomial separation is already impossible: choose $C = d = 1$ and use $m = n$. The theorem `distinct_pair_version_false` permits separation in either direction and still refutes it. Fixedness, explicitness and naturality of the family cannot rescue an assertion that fails for every possible family on this particular admissible pair.

The Lean record of a semantic proof system does not itself include a machine model or polynomial-time field. Its semantic universal assertion is not claimed to be definitionally the original conjecture. Application to the original domain is justified by the concrete polynomial-time admissibility proof in the preceding section, not by enlarging the universal domain.

5 Verification and reproduction

The project imports only Lean's `Std`, pinned to Lean 4.31.0. Build with `lake build`; audit every theorem using `lake env lean Audit.lean`; independently replay the compiled kernel objects with separate calls `lake env leanchecker Main` and `lake env leanchecker Costs`. The audited dependencies are only the usual foundational axioms `propext`, `Classical.choice`, and `Quot.sound`. There are no admitted proofs, custom axioms, native-decide calls, or numerical certificates standing in for an unbounded argument.

Sources. Official bilingual statement: TLMC conjecture 00000003980. For the standard definition of polynomial-time surjective proof systems, see Jan Krajíček, *The Cook–Reckhow definition*, Definition 1.1: Author-hosted PDF. No external theorem is required for the counterexample.